Análise do impacto da ventania de 07/11/2026 sobre a manutenção de árvores em Curitiba

Contexto

A cidade de Curitiba foi afetada por uma forte ventania no dia 07/11/2026, o que gerou uma série de ocorrências relacionadas a queda de árvores em vias públicas, bloqueio de passagem e riscos à mobilidade urbana. Nesse cenário, o estudo busca avaliar se a prefeitura já havia sido alertada previamente sobre riscos de poda em locais que posteriormente sofreram quedas durante o evento climático.

A análise parte da hipótese de que parte dos incidentes poderia ter sido evitada por meio de manutenção preventiva em árvores que já estavam em fila de atendimento, mas que não foram atendidas antes da tempestade. Com isso, o trabalho investiga a relação entre pedidos de poda em aberto antes do evento e as quedas registradas logo após a ventania.

Objetivo da análise

O objetivo geral deste trabalho é verificar a existência de sobreposição entre:

- pedidos de poda de árvores em aberto antes da ventania;
- registros de árvores caídas após o evento;
- regiões, bairros e ruas com maior concentração de alertas ignorados.

Mais especificamente, busca-se responder se a ocorrência de quedas de árvores em determinadas ruas e bairros estava associada a pedidos anteriores de manutenção preventiva que não foram atendidos.

Perguntas de pesquisa

1. Quantos pedidos de poda em árvores estavam abertos antes da ventania?
2. Quantas ocorrências de queda de árvore foram registradas após o evento?
3. Quantas dessas ocorrências ocorreram em ruas que já possuíam pedidos de poda em aberto?
4. Quantas ruas únicas foram afetadas por esse passivo?
5. Em quais regiões e bairros esse padrão foi mais concentrado?
6. O cruzamento entre pedidos de poda e quedas evidencia negligência na manutenção preventiva?

Base de dados

A análise foi realizada com dados do sistema de atendimento 156 da Prefeitura de Curitiba, considerando:

- solicitações relacionadas ao assunto "Árvore";
- pedidos de poda em aberto antes do dia 07/11/2026;
- chamados de queda ou galho caído após a ventania;
- dados geográficos e de bairro/regional para comparar os registros espacialmente.

A base foi tratada para padronizar nomes de logradouros, remover ruídos textuais, separar pedidos antes e depois do evento e comparar ocorrências por rua e bairro.

Metodologia

A metodologia adotada inclui:

- filtragem dos registros por assunto e período;
- separação da base em dois grupos: antes e depois da ventania;
- normalização de nomes de logradouros para permitir comparação;
- cruzamento entre ruas com pedido de poda e ruas com quedas de árvore;
- agregação por regional e bairro para identificar os locais mais afetados;
- análise geoespacial com cruzamento entre os dados do 156 e a malha viária da cidade.

Indicadores principais

Ao longo do estudo, serão utilizados indicadores como:

- total de pedidos de poda em aberto;
- total de ocorrências de queda após a ventania;
- número de ruas com interseção entre alerta prévio e queda;
- número de ruas únicas afetadas;
- distribuição por regional e bairro;
- mapeamento dos trechos urbanos com maior risco de impacto.

Importância do estudo

Este trabalho tem relevância tanto técnica quanto institucional, pois permite quantificar o impacto do atraso na manutenção preventiva de árvores e demonstrar, com base em dados públicos, o potencial efeito de decisões de manutenção e priorização de serviços. Os resultados podem subsidiar relatórios de gestão, ações preventivas e estratégias de manutenção urbana em áreas de risco.

Conclusão preliminar

A análise sugere que uma parcela relevante dos eventos de queda de árvores ocorreu em

locais que já haviam sido sinalizados previamente à prefeitura por meio de pedidos de poda. Esse padrão indica a existência de um passivo de manutenção que pode ter contribuído diretamente para o agravamento dos impactos da ventania. A continuidade da análise se concentra em mensurar esse efeito por região, bairro e rua, buscando identificar a extensão real do risco não mitigado antes do evento.

Código de análise

In [29]:
```
# importar pacotes necessários
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import unicodedata
import geopandas as gpd
import folium
```

Limpeza e filtragem dos dados de interesse

In [30]:
```
# Abrindo arquivo csv e limpeza coluna vazia
df = pd.read_csv("2026-11-08_156_-_Base_de_Dados.csv", sep = ";", encoding = "ut
df.drop(columns=['Unnamed: 11'], inplace=True)

# Selecionando apenas pedidos envolvidos com assunto "Árvore" para análise
df_arvores = df[df['Assunto'] == 'Árvore']
```

```
C:\Users\letre\AppData\Local\Temp\ipykernel_16648\437942262.py:3: Pandas4Warning:
The inplace keyword in DataFrame.drop is deprecated and will be removed in a futu
re version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace f
or more details.
  df.drop(columns=['Unnamed: 11'], inplace=True)
```

Separando pedidos por data, sendo antes e após ventania dia 07/11/2026

In [31]:
```
# Converter para datetime e isolar os dados do ano de 2026
df_arvores['DataCriacao'] = pd.to_datetime(df_arvores['DataCriacao'], format='%d
df_arvores = df_arvores[df_arvores['DataCriacao'].dt.year == 2026]

# Separar dataset por data usando .copy() para evitar o erro de memória do Panda
data_vendaval = pd.to_datetime('07/11/2026', format='%d/%m/%Y')
df_arvores_antes = df_arvores[df_arvores['DataCriacao'] < data_vendaval].copy()
df_arvores_depois = df_arvores[df_arvores['DataCriacao'] >= data_vendaval].copy(

# Filtrar DEPOIS: apenas quedas com ou sem bloqueio
df_arvores_depois = df_arvores_depois[df_arvores_depois["Subdivisao"].isin([
    "Árvore ou galho caído – com bloqueio de passagem",
    "Árvore ou galho caído – sem bloqueio de passagem"
])]

# Filtrar ANTES: apenas situações "Aberto" E subdivisões corretas
df_arvores_antes = df_arvores_antes[df_arvores_antes["Situacao"] == "Aberto"]
df_arvores_antes = df_arvores_antes[df_arvores_antes["Subdivisao"].isin([
    "Poda de manutenção em árvores",
    "Avaliação para corte em via pública"
```

```
])]

# Guardar com os nomes EXATOS que serão lidos na próxima etapa
df_arvores_depois.to_csv('df_arvores_depois.csv', index=False, sep=';', encoding
df_arvores_antes.to_csv('df_arvores_antes_aberto.csv', index=False, sep=';', enc

print("Filtros aplicados e ficheiros CSV guardados com sucesso!")
```

```
Filtros aplicados e ficheiros CSV guardados com sucesso!
```

Limpeza de lougradouro para possibilitar avaliação de sobreposição

Limpeza posterior à filtragem para mais rápido processamento

In [32]:

```
# Carregar as bases filtradas
df_antes = pd.read_csv('df_arvores_antes.csv', sep=';', encoding='utf-8')
df_depois = pd.read_csv('df_arvores_depois.csv', sep=';', encoding='utf-8')

# Criar uma coluna sinalizadora para facilitar a separação futura
df_antes['grupo_analise'] = 'antes_poda'
df_depois['grupo_analise'] = 'depois_queda'

# Concatenar os datasets
df_arvores = pd.concat([df_antes, df_depois], ignore_index=True)

# Função de limpeza do Logradouro
def limpar_logradouro(endereco):
    if pd.isna(endereco):
        return ""

    # Maiúsculas e remoção de acentos
    rua = str(endereco).upper().strip()
    rua = ''.join(c for c in unicodedata.normalize('NFD', rua) if unicodedata.ca

    # Remover prefixos comuns de Curitiba
    prefixos = [r'^RUA\s+', r'^R\s+', r'^AVENIDA\s+', r'^AV\s+', r'^TRAVESSA\s+'
    for prefixo in prefixos:
        rua = re.sub(prefixo, '', rua)

    # Limpar pontuações residuais substituindo por espaço
    rua = re.sub(r'[.,;-]', ' ', rua)

    # Remover espaços duplos acidentais
    rua = re.sub(r'\s+', ' ', rua).strip()

    return rua

# Aplicar a função diretamente na coluna Logradouro
df_arvores['Logradouro_Limpo'] = df_arvores['Logradouro'].apply(limpar_logradour

# Visualizar o resultado da limpeza
display(df_arvores[['Logradouro', 'Logradouro_Limpo', 'Bairro', 'grupo_analise']
```

	Logradouro	Logradouro_Limpo	Bairro	grupo_analise
0	Rua Antônio Boza	ANTONIO BOZA	SITIO CERCADO	antes_poda
1	Valdomiro Silveira	VALDOMIRO SILVEIRA	BOA VISTA	antes_poda
2	Rio Norte Belém	RIO NORTE BELEM	SAO LOURENCO	antes_poda
3	RUA ADA MACAGGI	ADA MACAGGI	BAIRRO ALTO	antes_poda
4	Presidente Taunay	PRESIDENTE TAUNAY	BATEL	antes_poda

Análise de sobreposição de dados - cruzamento das bases

In [33]:
```
#  Separar os datasets limpos utilizando a coluna sinalizadora
df_poda = df_arvores[df_arvores['grupo_analise'] == 'antes_poda'].copy()
df_queda = df_arvores[df_arvores['grupo_analise'] == 'depois_queda'].copy()

#  Realizar o cruzamento (Inner Join) por Rua e Bairro - os registros onde a mes
df_cruzamento = pd.merge(
    df_poda,
    df_queda,
    on=['Logradouro_Limpo', 'Bairro'],
    suffixes=('_poda', '_queda'),
    how='inner'
)

#  Remover duplicatas para descobrir exatamente quantas ruas únicas foram afetad
ruas_alerta_ignorado = df_cruzamento[['Logradouro_Limpo', 'Bairro']].drop_duplic

# Calculo de porcentagem de ruas afetadas que já tinham alerta de poda
total_ruas_quedas = len(df_queda[['Logradouro_Limpo', 'Bairro']].drop_duplicates
total_ruas_alerta_ignorado = len(ruas_alerta_ignorado)
porcentagem_alerta_ignorado = (total_ruas_alerta_ignorado / total_ruas_quedas) *


# Exibir os resultados finais que respondem às perguntas do impacto da ventania

print("--- RESULTADOS DA ANÁLISE DE IMPACTO ---")
print(f"1. Total de pedidos de poda em aberto (antes de 07/11): {len(df_poda)}")
print(f"2. Total de chamados de queda (após a ventania de 07/11): {len(df_queda)}
print(f"3. Número de ocorrências de quedas em locais que JÁ TINHAM pedido de pod
print(f"4. Total de RUAS ÚNICAS afetadas por esse passivo: {len(ruas_alerta_igno
print(f"5. Porcentagem de ruas afetadas que JÁ TINHAM alerta de poda: {porcentag
```

```
--- RESULTADOS DA ANÁLISE DE IMPACTO ---
1. Total de pedidos de poda em aberto (antes de 07/11): 14756
2. Total de chamados de queda (após a ventania de 07/11): 741
3. Número de ocorrências de quedas em locais que JÁ TINHAM pedido de poda: 5090
4. Total de RUAS ÚNICAS afetadas por esse passivo: 253
5. Porcentagem de ruas afetadas que JÁ TINHAM alerta de poda: 77.85%
```

Representações Gráficas

Mapa interativo

In [34]:
```
# Funções de Limpeza
def limpar_nome_universal(nome):
    if pd.isna(nome): return ""
    texto = str(nome).upper()
    texto = ''.join(c for c in unicodedata.normalize('NFD', texto) if unicodedat
    texto = re.sub(r'[.,;-]', ' ', texto)
    texto = re.sub(r'\s+', ' ', texto).strip()
    texto = re.sub(r'^(RUA|R|AVENIDA|AV|TRAVESSA|TR|PRACA|PC|ALAMEDA|AL|RODOVIA|
    return texto.strip()

#  Função dedicada aos Bairros
def limpar_bairro(nome):
    if pd.isna(nome): return ""
    texto = str(nome).upper()
    texto = ''.join(c for c in unicodedata.normalize('NFD', texto) if unicodedat
    return re.sub(r'\s+', ' ', texto).strip()

#  Carregar e Preparar o Shapefile
gdf_ruas = gpd.read_file('EIXO_RUA/EIXO_RUA.shp')

# Combinar nome da rua e limpar
gdf_ruas['Nome_Combinado'] = gdf_ruas['NMVIA'].fillna(gdf_ruas['NMVIA_ANT']).fil
gdf_ruas['Nome_Limpo_Shape'] = gdf_ruas['Nome_Combinado'].apply(limpar_nome_univ

# Limpar bairros da Esquerda e Direita do Shapefile
gdf_ruas['BAIRRO_E_Limpo'] = gdf_ruas['BAIRRO_E'].apply(limpar_bairro)
gdf_ruas['BAIRRO_D_Limpo'] = gdf_ruas['BAIRRO_D'].apply(limpar_bairro)

# Preparar as bases do 156 INCLUINDO o Bairro
ruas_poda = df_poda[['Logradouro_Limpo', 'Bairro']].drop_duplicates().copy()
ruas_poda['Logradouro_Limpo'] = ruas_poda['Logradouro_Limpo'].apply(limpar_nome_
ruas_poda['Bairro_Limpo'] = ruas_poda['Bairro'].apply(limpar_bairro)

ruas_queda = df_queda[['Logradouro_Limpo', 'Bairro']].drop_duplicates().copy()
ruas_queda['Logradouro_Limpo'] = ruas_queda['Logradouro_Limpo'].apply(limpar_nom
ruas_queda['Bairro_Limpo'] = ruas_queda['Bairro'].apply(limpar_bairro)

ruas_cruzamento = df_cruzamento[['Logradouro_Limpo', 'Bairro']].drop_duplicates(
ruas_cruzamento['Logradouro_Limpo'] = ruas_cruzamento['Logradouro_Limpo'].apply(
ruas_cruzamento['Bairro_Limpo'] = ruas_cruzamento['Bairro'].apply(limpar_bairro)

# 4. Cruzamento Vetorial (Nome da Rua + Validação do Bairro)
# PODA
gdf_poda_mapa = gdf_ruas.merge(ruas_poda, left_on='Nome_Limpo_Shape', right_on='
mask_poda = (gdf_poda_mapa['Bairro_Limpo'] == gdf_poda_mapa['BAIRRO_E_Limpo']) |
gdf_poda_mapa = gdf_poda_mapa[mask_poda]

# QUEDA
gdf_queda_mapa = gdf_ruas.merge(ruas_queda, left_on='Nome_Limpo_Shape', right_on
mask_queda = (gdf_queda_mapa['Bairro_Limpo'] == gdf_queda_mapa['BAIRRO_E_Limpo']
gdf_queda_mapa = gdf_queda_mapa[mask_queda]

# CRUZAMENTO (ALERTAS IGNORADOS)
gdf_cruzamento_mapa = gdf_ruas.merge(ruas_cruzamento, left_on='Nome_Limpo_Shape'
mask_cruzamento = (gdf_cruzamento_mapa['Bairro_Limpo'] == gdf_cruzamento_mapa['B
gdf_cruzamento_mapa = gdf_cruzamento_mapa[mask_cruzamento]

# . Gerar o Mapa
mapa_curitiba = folium.Map(
    location=[-25.4284, -49.2733],
```

```
    zoom_start=12,
    tiles='https://server.arcgisonline.com/ArcGIS/rest/services/Canvas/World_Lig
    attr='Tiles &copy; Esri &mdash; Esri, DeLorme, NAVTEQ'
)

# Adicionar Camada AZUL
if not gdf_poda_mapa.empty:
    folium.GeoJson(
        gdf_poda_mapa,
        name='1. Pedidos de Poda (Antes 07/11)',
        style_function=lambda x: {'color': 'blue', 'weight': 2, 'opacity': 0.4},
        tooltip=folium.GeoJsonTooltip(fields=['Nome_Combinado', 'Bairro_Limpo'],
    ).add_to(mapa_curitiba)

# Adicionar Camada VERMELHA
if not gdf_queda_mapa.empty:
    folium.GeoJson(
        gdf_queda_mapa,
        name='2. Quedas de Árvores (Após 07/11)',
        style_function=lambda x: {'color': 'red', 'weight': 2, 'opacity': 0.7},
        tooltip=folium.GeoJsonTooltip(fields=['Nome_Combinado', 'Bairro_Limpo'],
    ).add_to(mapa_curitiba)

# Adicionar Camada ROXA
if not gdf_cruzamento_mapa.empty:
    folium.GeoJson(
        gdf_cruzamento_mapa,
        name='3. Alertas Ignorados (Poda + Queda)',
        style_function=lambda x: {'color': 'purple', 'weight': 5, 'opacity': 1.0
        tooltip=folium.GeoJsonTooltip(fields=['Nome_Combinado', 'Bairro_Limpo'],
    ).add_to(mapa_curitiba)

folium.LayerControl().add_to(mapa_curitiba)
mapa_curitiba.save('mapa_alertas_curitiba.html')
```

Representação em funil de negligência

In [ ]:

```
import matplotlib.patches as patches

fig, ax = plt.subplots(figsize=(8, 10), facecolor='#FAFAFA')
ax.axis('off')
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)

# Títulos
plt.text(0.5, 0.92, "O FUNIL DA NEGLIGÊNCIA", fontsize=24, fontweight='bold', ha
plt.text(0.5, 0.86, "Como o atraso na manutenção preventiva\npotencializou o imp

# Box 1 (Top) - O Passivo
rect1 = patches.FancyBboxPatch((0.1, 0.68), 0.8, 0.12, boxstyle="round,pad=0.02,
ax.add_patch(rect1)
plt.text(0.5, 0.74, "14.756", fontsize=38, fontweight='bold', ha='center', color
plt.text(0.5, 0.70, "Pedidos de poda aguardando ação", fontsize=12, fontweight='

# Seta 1
plt.arrow(0.5, 0.65, 0, -0.06, head_width=0.04, head_length=0.03, fc='#BDC3C7',

# Box 2 (Middle) - O Impacto
rect2 = patches.FancyBboxPatch((0.15, 0.43), 0.7, 0.13, boxstyle="round,pad=0.02
```

```
ax.add_patch(rect2)
plt.text(0.5, 0.50, "741", fontsize=38, fontweight='bold', ha='center', color='w
plt.text(0.5, 0.45, "Locais com árvores caídas na ventania", fontsize=12, fontwe

# Seta 2
plt.arrow(0.5, 0.40, 0, -0.06, head_width=0.04, head_length=0.03, fc='#BDC3C7',

# Box 3 (Bottom) - A Negligência
rect3 = patches.FancyBboxPatch((0.15, 0.16), 0.7, 0.15, boxstyle="round,pad=0.02
ax.add_patch(rect3)
# Subimos ligeiramente o 253 e o texto para dar espaço ao selo
plt.text(0.5, 0.25, "253", fontsize=38, fontweight='bold', ha='center', color='w
plt.text(0.5, 0.19, "Ruas atingidas já tinham alerta na prefeitura", fontsize=12

# --- DESTAQUE DA PORCENTAGEM  ---
plt.text(0.5, 0.11, " Isso representa 78% das ruas afetadas (Quase 8 em cada 10)
         fontsize=11, fontweight='bold', ha='center', va='center', color='#C0392
         bbox=dict(boxstyle='round,pad=0.6,rounding_size=0.8', facecolor='#F1C40

# Fonte
plt.text(0.5, 0.04, "Fonte: Central 156 / IPPUC - Análise de Dados (Nov/2026)",

plt.savefig('infografico_funil.png', dpi=300, bbox_inches='tight')
plt.show()
```



Lougradores mais ignorados

In [36]:
```
# Agrupar os dados para descobrir qual rua teve o maior cruzamento de alertas ig
ranking_ruas = df_cruzamento.groupby(['Logradouro_Limpo', 'Bairro']).size().rese
ranking_ruas = ranking_ruas.sort_values(by='Intensidade_Negligencia', ascending=

# Criar uma coluna legível unindo Rua e Bairro para o eixo do gráfico
ranking_ruas['Endereco_Completo'] = ranking_ruas['Logradouro_Limpo'] + '\n(' + r

# Configurar o visual da imagem (Fundo limpo)
fig, ax = plt.subplots(figsize=(10, 6), facecolor='#FAFAFA')
sns.set_theme(style="white")
ax.set_facecolor('#FAFAFA')

# Vermelho Alerta para o 1º lugar, Cinza Neutro para os restantes
```

```
cores_destaque = ['#E74C3C'] + ['#BDC3C7'] * (len(ranking_ruas) - 1)

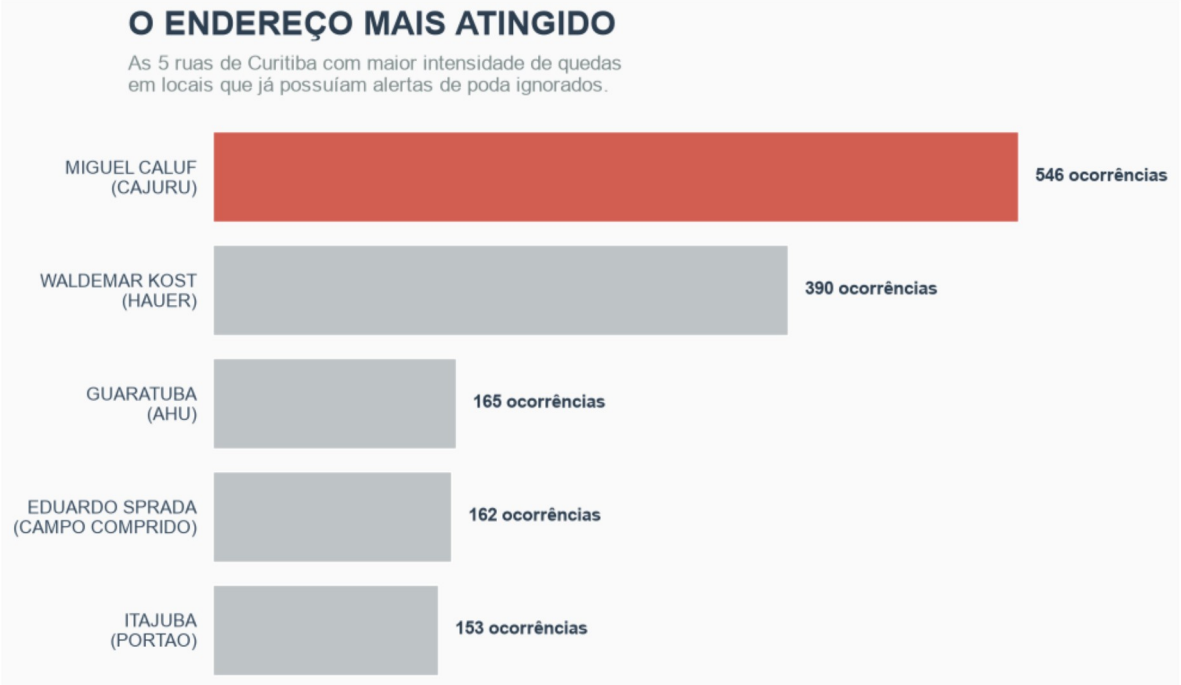
# Desenhar as barras
barras = sns.barplot(
    data=ranking_ruas,
    x='Intensidade_Negligencia',
    y='Endereco_Completo',
    palette=cores_destaque,
    hue='Endereco_Completo',
    legend=False
)

# Colocar os números no final de cada barra
for p in barras.patches:
    ax.annotate(f"{int(p.get_width())} ocorrências",
                (p.get_width(), p.get_y() + p.get_height() / 2.),
                xytext=(10, 0), textcoords='offset points',
                va='center', fontsize=11, fontweight='bold', color='#2C3E50')

# Limpeza do Design (Remover eixos e bordas)
sns.despine(left=True, bottom=True)
ax.set(xticklabels=[], xlabel='', ylabel='')
ax.tick_params(axis='y', labelsize=11, colors='#34495E', left=False)

# Títulos
plt.text(-0.1, 1.15, "O ENDEREÇO MAIS ATINGIDO", fontsize=20, fontweight='heavy'
plt.text(-0.1, 1.05, "As 5 ruas de Curitiba com maior intensidade de quedas\nem

# Salvar a imagem
plt.tight_layout()
plt.savefig('grafico_pior_rua.png', dpi=300, bbox_inches='tight', facecolor='#FA
plt.show()
```



Localização das ruas negligênciadas

In [ ]:
```
# Configurar o tamanho e estilo da figura
fig, ax = plt.subplots(figsize=(12, 12), facecolor='#FAFAFA')
ax.set_facecolor('#FAFAFA')
```

```
ax.axis('off') # Remove os eixos de latitude e longitude

# Desenhar a malha viária completa de Curitiba (Fundo cinza claro)
gdf_ruas.plot(ax=ax, color='#c7c7c7', linewidth=0.5, alpha=0.8)

# Sobrepor os trechos de negligência (O cruzamento exato em roxo/vermelho escuro
if not gdf_cruzamento_mapa.empty:
    gdf_cruzamento_mapa.plot(ax=ax, color='#bd1c1c', linewidth=2.5, alpha=1.0)

# Título
plt.text(0.5, 1.01, "A localização exata dos trechos urbanos onde a prefeitura i
         fontsize=14, ha='center', color='#7F8C8D', transform=ax.transAxes)

# Fonte e Créditos
plt.text(0.5, -0.02, "Fonte: Central 156 / IPPUC - Análise Geoespacial Independe
         fontsize=10, ha='center', color='#95A5A6', transform=ax.transAxes)

# Salvar em altíssima resolução
plt.savefig('mapa_estatico.png', dpi=300, bbox_inches='tight', facecolor='#FAFAF

# Exibir no notebook
plt.show()
```

A localização exata dos trechos urbanos onde a prefeitura ignorou os alertas preventivos e as árvores caíram durante a ventania.



Fonte: Central 156 / IPPUC - Análise Geoespacial Independente (Nov/2026)